The testing gem shoulda-matchers defines matchers that specifically test for the existence of validations and associations,[3] like so:

```
describe Task do
  it { should belong_to(:project) }
  it { should belong_to(:user) }
  it { should ensure_length_of(:name) }
end
```

Tests like that are not particularly valuable for a TDD process because they aren't about the design of new features. If you're doing the TDD process, you shouldn't start from the idea that your Task belongs to a Project. Rather, as you describe features, the relationship is implied from the feature tests you're writing. More operationally, this means that in a good TDD process, any condition in the code that would cause a direct test like those Shoulda matchers to fail would also cause another test to fail. In which case, what's the point of the Shoulda matcher?

To briefly and halfheartedly argue the other side, you often don't need to go through a whole TDD process to know that a relationship or a validation should exist, and these tests don't cost very much to write. And to rebut myself, part of doing the TDD process is to force you to examine the things you think you know and prove that they are really necessary. In particular, if I see a lot of those should belong_to lines of code in a test suite, rightly or wrongly I'm going to be worried about the test suite's effectiveness.

Validations are a little different since the validity of a piece of data is often a legitimate and complex piece of business logic and the Rails helpers cover only a set of common cases. I think you can make validations in your code work efficiently with the TDD process if you focus on the functional part—"this object is or is not valid"—rather than the "I'm using a Rails numerical validator" part. Also, consider using a Rails database-blocking validation only as a last resort when you sincerely believe it's worth potentially raising an exception rather than having this data in your database.

I would typically test for the functional effect of an invalid object rather than the implementation fact of the existence of a Rails validation. The functional effect is often along the lines of an object or objects not saving to the database. If you wanted to augment your project tracker's project creation to require that all the tasks for the new project have a size, you might try something like this:

3. https://github.com/thoughtbot/shoulda-matchers